

InventoryHub Security, Testing & Compliance Report

Phases 3 & 4: Secure Codebase, Testing, and Compliance Evidence

Table of Contents

InventoryHub Security, Testing & Compliance Report.....	1
1. Introduction.....	3
2. Project Overview.....	3
3. Defensive Design and Security Implementations	3
3.1 HTTP Header Hardening	4
3.2 Cross-Origin Resource Sharing Protection.....	4
3.3 Request Payload Size Restriction.....	4
3.4 Rate Limiting.....	4
4. Authentication and Role-Based Access Control	5
4.1 Password Security	5
4.2 JWT-Based Authentication	5
4.3 Role-Based Access Control Matrix	5
5. Audit Logging and Security Alerting	6
5.1 Audit Logging.....	6
5.2 Security Alert Triggers	7
6. External Security Integrations: Redis and Kafka	7
6.1 Redis Integration	7
6.2 Kafka Integration	7
6.3 Fallback Handling.....	8
7. Static Application Security Testing.....	8
7.1 GitHub CodeQL	8
7.2 SonarQube	10

8. Dynamic Application Security Testing and Pentest Validation	10
8.1 OWASP ZAP API Scanning.....	11
8.2 Manual Pentest Scenario Matrix	13
9. Docker and Kubernetes Security Hardening	14
9.1 Dockerfile Security	14
9.2 Production Dependency Installation	15
9.3 Kubernetes Security Context	15
9.4 Kubernetes Secrets Management.....	16
9.5 Kubernetes Network Policy.....	16
10. CIS Docker and Kubernetes Compliance Checklist.....	16
11. Jest Integration Test Results.....	17
12. Screenshot Evidence	18
Jest Test Pass Output	18
Input Validation Blocking XSS Payload	20
Staff Role Blocked from Product Deletion	21
Admin Security Alerts and Audit Logs Dashboard	23
Trivy Container Vulnerability Scan.....	23
Kubernetes Resources and Network Policy Status	24
13. Conclusion	26

1. Introduction

This report presents the security implementation, testing, and compliance work completed for **Phases 3 and 4** of the InventoryHub Secure Software Development project. The main objective of these phases was to strengthen the application codebase, implement secure development practices, verify security controls through testing, and provide evidence of compliance with container and Kubernetes security standards.

InventoryHub is a Node.js and Express-based inventory management system designed with security controls at the application, container, and deployment levels. The system includes authentication, role-based access control, input validation, audit logging, security alerting, container hardening, Kubernetes network isolation, and automated testing.

This report documents the implemented security mechanisms, static and dynamic testing activities, container orchestration hardening, and verification evidence required for final submission.

2. Project Overview

InventoryHub is a secure inventory management application that allows authorized users to manage product records while enforcing strong access control and monitoring mechanisms. The application supports two major user roles: **admin** and **staff**.

The admin role has full access to product management, user registration, audit logs, and security alerts. The staff role has limited access and is restricted from performing high-risk actions such as deleting products or viewing security logs.

The application uses multiple security-focused technologies and practices, including:

- Node.js and Express for backend services.
- MongoDB for persistent data storage.
- JWT-based authentication for protected routes.
- bcryptjs for secure password hashing.
- Zod validation for input sanitization.
- Helmet for HTTP security headers.
- CORS restrictions for cross-origin access control.
- Redis for brute-force login tracking.
- Kafka for security event streaming.
- Docker for containerized deployment.
- Kubernetes for orchestration and network policy enforcement.
- Jest for automated integration testing.
- CodeQL and SonarQube for static code analysis.
- OWASP ZAP and manual pentest scenarios for dynamic testing.

3. Defensive Design and Security Implementations

The InventoryHub application uses a layered defensive design to reduce exposure to common web application attacks. Security controls are applied at the request-processing layer before data reaches sensitive business logic or the database.

The Express server follows a sequential middleware pipeline where requests are filtered, validated, authenticated, authorized, logged, and then processed. This approach improves security because invalid or unauthorized requests are blocked early.

3.1 HTTP Header Hardening

The application uses **Helmet** middleware to apply secure HTTP response headers. These headers reduce browser-based attack risks such as clickjacking, MIME sniffing, cross-site scripting exposure, and unnecessary referrer leakage.

Important header protections include:

Security Header	Purpose
X-Frame-Options: SAMEORIGIN	Prevents the application from being embedded in unauthorized third-party iframes.
X-Content-Type-Options: nosniff	Stops browsers from guessing MIME types incorrectly.
Referrer-Policy: no-referrer	Prevents sensitive URL information from being leaked through referrer headers.

These headers strengthen the client-side security posture of the application.

3.2 Cross-Origin Resource Sharing Protection

The backend uses a restricted CORS configuration:

```
cors({ origin: CLIENT_ORIGIN })
```

This ensures that only the approved frontend origin can send browser-based requests to the API. It helps prevent unauthorized websites from interacting with protected API routes through a victim's browser session.

3.3 Request Payload Size Restriction

To reduce the risk of denial-of-service attacks using oversized request bodies, the application limits incoming JSON payloads:

```
express.json({ limit: '10kb' })
```

This prevents attackers from sending excessively large payloads that may consume memory or degrade server performance.

3.4 Rate Limiting

Rate limiting is implemented to control repeated requests from the same IP address.

API Area	Limit
General API routes	100 requests per 15 minutes per IP
Authentication routes	20 attempts per 15 minutes per IP

This helps protect against:

- Brute-force login attempts.
- Automated credential guessing.
- API scraping.
- Repeated abuse of sensitive endpoints.

4. Authentication and Role-Based Access Control

Authentication and authorization are central security features of InventoryHub. The system ensures that only verified users can access protected resources, and only users with the correct role can perform privileged actions.

4.1 Password Security

User passwords are never stored in plain text. The application uses **bcryptjs** with a salt factor of 10 to securely hash passwords before storing them in the database.

This protects user credentials even if the database is exposed, because attackers cannot directly read the original passwords.

4.2 JWT-Based Authentication

Protected API routes require a valid JWT Bearer token in the request header. The application verifies the token before allowing access to protected resources.

The system supports:

- Locally signed JWT tokens using `JWT_SECRET`.
- Federated OIDC-style tokens verified through RSA-256 public key validation.

This design supports both local authentication and mock identity provider integration.

4.3 Role-Based Access Control Matrix

The application uses role-based access control to separate admin and staff privileges.

Feature / API Route	Allowed Roles	Staff Access	Admin Access	Response on Violation
---------------------	---------------	--------------	--------------	-----------------------

User Registration	Admin	Blocked	Allowed	403 Forbidden and audit alert
Product Query	Admin, Staff	Allowed	Allowed	401 Unauthorized if token missing
Product Create	Admin, Staff	Allowed	Allowed	Zod validation applied
Product Update	Admin, Staff	Allowed	Allowed	Stock deviation checks applied
Product Delete	Admin	Blocked	Allowed	403 Forbidden and anomaly alert
Security Alerts	Admin	Blocked	Allowed	403 Forbidden and security alert
Audit Logs	Admin	Blocked	Allowed	403 Forbidden and security alert

This access matrix ensures that staff users can perform routine inventory operations but cannot access administrative or destructive features.

5. Audit Logging and Security Alerting

InventoryHub records important user actions and security-related events. This provides accountability, supports incident investigation, and helps detect suspicious activity.

5.1 Audit Logging

Audit logs are written to MongoDB whenever important actions occur. These include:

- Successful login attempts.
- Failed login attempts.
- Product creation.
- Product updates.
- Product deletions.
- Unauthorized access attempts.
- Input validation violations.
- Access to protected admin endpoints.

Each audit log may include:

- User ID or username.
- Requested API path.
- Event type.
- Timestamp.
- Client IP address.
- Action result.

Audit logging improves traceability and allows administrators to understand what occurred inside the system.

5.2 Security Alert Triggers

Security alerts are generated when suspicious or high-risk patterns are detected.

Alert Type	Severity	Trigger Condition
BRUTE_FORCE_ATTEMPT	Critical	More than 5 failed login attempts within 10 minutes
STOCK_CHANGE_EXCEEDED	Medium / High	Product stock created or changed by more than 100 units
UNAUTHORIZED_DELETION_ATTEMPT	High	Staff user attempts to delete a product
REPEATED_DELETIONS	Critical	More than 3 products deleted within 1 minute

These alerts help administrators quickly identify abnormal behavior and potential attacks.

6. External Security Integrations: Redis and Kafka

InventoryHub uses Redis and Kafka to improve security monitoring, event handling, and resilience.

6.1 Redis Integration

Redis is used to track failed login attempts. Failed login counters are stored with keys such as:

```
failed_login:<username>
```

Each failed login record has a 10-minute expiry window. If repeated failures exceed the configured threshold, the system triggers a brute-force security alert.

Redis improves the efficiency of temporary security counters because it is fast and supports automatic key expiration.

6.2 Kafka Integration

Kafka is used to stream security events to the `inventory-events` topic. Examples of streamed events include:

```
auth.login_failed
security.unauthorized_access
product.stock_changed
security.deletion_attempt
```

Kafka provides an event-driven mechanism for future monitoring, alerting, and centralized logging systems.

6.3 Fallback Handling

The application is designed to remain functional even if Redis or Kafka becomes unavailable. If these external services are disconnected, the application falls back to MongoDB-based storage and continues recording security-relevant information.

This improves service availability and avoids complete failure when supporting infrastructure is temporarily unavailable.

7. Static Application Security Testing

Static Application Security Testing was performed to identify security weaknesses in the code before runtime. Two tools were configured: **GitHub CodeQL** and **SonarQube**.

7.1 GitHub CodeQL

GitHub CodeQL is configured through the workflow file:

```
.github/workflows/codeql.yml
```

CodeQL runs automatically on:

- Push events to the main branch.
- Pull requests targeting the main branch.
- Weekly scheduled scans.

The CodeQL configuration focuses on JavaScript and TypeScript analysis. It helps detect vulnerabilities such as:

- Unsafe data flow.
- Injection risks.
- Insecure API usage.
- Untrusted input reaching sensitive functions.
- Common coding weaknesses.

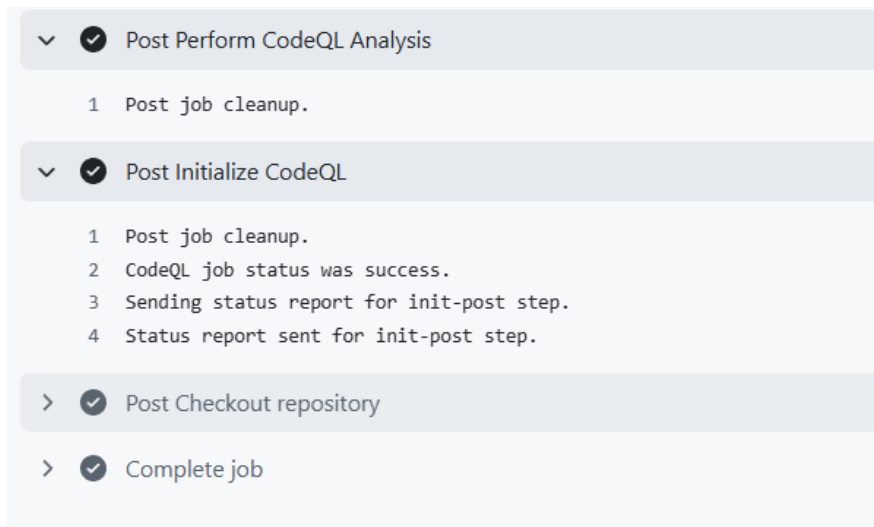
The workflow is also configured with `security-events: write` permission so results can be uploaded to the repository security alerts section.

Initialize CodeQL

```
1 ▶ Run github/codeql-action/init@v3
11 Warning: CodeQL Action v3 will be deprecated in December 2026. Please update all occurrences of the CodeQL Action in your workflow files to v4. For more information, see
    https://github.blog/changelog/2025-10-28-upcoming-deprecation-of-codeql-action-v3/
12 Job run UUID is 7ac74123-13f8-4e16-b6ae-686edd7cb16a.
13 ▶ Setup CodeQL tools
72 ▶ Validating workflow
474 ▶ Load language configuration
855 While resolving threads, found a cgroup CPUs file with 4 CPUs in /sys/fs/cgroup/cpuset.cpus.effective.
856 Writing augmented user configuration file to /home/runner/work/_temp/user-config.yaml
857 ▶ Augmented user configuration file contents
860 /opt/hostedtoolcache/CodeQL/2.25.6/x64/codeql/codeql database init --force-overwrite --db-cluster /home/runner/work/_temp/codeql_databases --source-root=/home/runner/work/Inventory-Management_SSDD-Final/Inventory-Management_SSDD-Final --calculate-language-specific-baseline --sublanguage-file-coverage --extractor-include-aliases --language-javascript --codescanning-config=/home/runner/work/_temp/user-config.yaml --overlay-base
861 Package install location: /home/runner/.codeql/packages
862 Nothing downloaded.
863 Calculating baseline information in /home/runner/work/Inventory-Management_SSDD-Final/Inventory-Management_SSDD-Final
864 Running command in /home/runner/work/Inventory-Management_SSDD-Final/Inventory-Management_SSDD-Final: [/opt/hostedtoolcache/CodeQL/2.25.6/x64/codeql/actions/tools/configure-baseline.sh]
865 Running command in /home/runner/work/Inventory-Management_SSDD-Final/Inventory-Management_SSDD-Final: [/opt/hostedtoolcache/CodeQL/2.25.6/x64/codeql/javascript/tools/configure-baseline.sh]
866 Calculated baseline information for languages: actions, javascript (77ms).
867 Resolving extractor javascript.
868 Successfully loaded extractor JavaScript/TypeScript (javascript) from /opt/hostedtoolcache/CodeQL/2.25.6/x64/codeql/javascript.
869 Created skeleton CodeQL database at /home/runner/work/_temp/codeql_databases/javascript. This in-progress database is ready to be populated by an extractor.
870 Recording pack reference codeql/javascript-queries at /opt/hostedtoolcache/CodeQL/2.25.6/x64/codeql/qlpacks/codeql/javascript-queries/2.3.11.
```

Perform CodeQL Analysis

```
1 ▶ Run github/codeql-action/analyze@v3
28 While resolving threads, found a cgroup CPUs file with 4 CPUs in /sys/fs/cgroup/cpuset.cpus.effective.
29 ▶ Generating diff range extension pack
31 ▶ Extracting javascript
332 ▶ Finalizing javascript
340 ▶ Running queries for javascript
893 CodeQL scanned 1 out of 1 GitHub Actions files and 3 out of 3 JavaScript files in this invocation. Check
    https://github.com/x0shariq0/Inventory-Management\_SSDD-Final/security/code-scanning/tools/CodeQL/status
894
895 Post-processing sarif files: ["/home/runner/work/Inventory-Management_SSDD-Final/results/javascript.sarif]
896 Adding fingerprints to SARIF file. See https://docs.github.com/en/code-security/reference/code-scanning
    information.
897 ▶ Uploading code scanning results
900 ▶ Cleaning up databases
908 Uploading overlay-base database to Actions cache with key codeql-overlay-base-database-1-673738e94a8102
909 /usr/bin/tar --posix -cf cache.tzst --exclude cache.tzst -P -C /home/runner/work/Inventory-Management_SSDD-Final/Inventory-Management_SSDD-Final --compress-program zstdmt
910 Sent 16301922 of 16301922 (100.0%), 28.8 MBs/sec
911 Successfully uploaded overlay-base database from /home/runner/work/_temp/codeql_databases
912 ▶ Cleaning up databases
918 /opt/hostedtoolcache/CodeQL/2.25.6/x64/codeql/codeql database bundle /home/runner/work/_temp/codeql_databases/javascript.zip --name=javascript --include base-databases
919 Creating bundle metadata for /home/runner/work/_temp/codeql_databases/javascript.zip...
920 Creating zip file at /home/runner/work/_temp/codeql_databases/javascript.zip.
921 ▶ Waiting for processing to finish
```



7.2 SonarQube

SonarQube is configured using:

```
sonar-project.properties
```

The analysis focuses on important source files such as:

```
server.js  
src/
```

Unnecessary directories are excluded from scanning:

```
node_modules/**  
coverage/**  
reports/**  
docs/**  
docker/**  
k8s/**
```

The expected quality gate requirements include:

Metric	Required Standard
New vulnerabilities	Zero
Security rating	A
Blocker bugs	Zero
Code quality	Must pass quality gate

SonarQube supports code quality improvement by identifying bugs, vulnerabilities, maintainability issues, and insecure patterns.

8. Dynamic Application Security Testing and Pentest Validation

Dynamic Application Security Testing was performed to validate the application while running. This included OWASP ZAP scanning and manual pentest scenarios.

8.1 OWASP ZAP API Scanning

OWASP ZAP was used to scan the running API surface. The scan tests exposed endpoints for common web vulnerabilities.

Example scan command:

```
docker run -t ghcr.io/zaproxy/zaproxy:stable zap-api-scan.py -t  
http://host.docker.internal:3000/api/products -f openapi
```

The purpose of the scan is to detect common issues such as:

- Missing security headers.
- Injection weaknesses.
- Improper access control.
- Unvalidated input.
- Insecure API behavior.

```
total of 31 URLs  
PASS: Directory Browsing [0]  
PASS: Vulnerable JS Library (Powered by Retire.js) [10003]  
PASS: Swagger UI Secret & Vulnerability Detector [100043]  
PASS: In Page Banner Information Leak [10009]  
PASS: Cookie No HttpOnly Flag [10010]  
PASS: Cookie Without Secure Flag [10011]  
PASS: Re-examine Cache-control Directives [10015]  
PASS: Cross-Domain JavaScript Source File Inclusion [10017]  
PASS: Content-Type Header Missing [10019]  
PASS: Anti-clickjacking Header [10020]  
PASS: X-Content-Type-Options Header Missing [10021]  
PASS: Information Disclosure - Debug Error Messages [10023]  
PASS: Information Disclosure - Sensitive Information in URL [10024]  
PASS: Information Disclosure - Sensitive Information in HTTP Referre  
PASS: HTTP Parameter Override [10026]  
PASS: Information Disclosure - Suspicious Comments [10027]  
PASS: Off-site Redirect [10028]  
PASS: Cookie Poisoning [10029]  
PASS: User Controllable Charset [10030]  
PASS: User Controllable HTML Element Attribute (Potential XSS) [1003
```

PASS: Off-site Redirect [10028]
PASS: Cookie Poisoning [10029]
PASS: User Controllable Charset [10030]
PASS: User Controllable HTML Element Attribute (Potential XSS) [10031]
PASS: Viewstate [10032]
PASS: Directory Browsing [10033]
PASS: Heartbleed OpenSSL Vulnerability (Indicative) [10034]
PASS: Strict-Transport-Security Header [10035]
PASS: HTTP Server Response Header [10036]
PASS: Server Leaks Information via "X-Powered-By" HTTP Response Header [10037]
PASS: Content Security Policy (CSP) Header Not Set [10038]
PASS: X-Backend-Server Header Information Leak [10039]
PASS: Secure Pages Include Mixed Content [10040]
PASS: HTTP to HTTPS Insecure Transition in Form Post [10041]
PASS: HTTPS to HTTP Insecure Transition in Form Post [10042]
PASS: User Controllable JavaScript Event (XSS) [10043]
PASS: Big Redirect Detected (Potential Sensitive Information Leak) [10044]
PASS: Source Code Disclosure - /WEB-INF Folder [10045]
PASS: HTTPS Content Available via HTTP [10047]
PASS: Remote Code Execution - Shell Shock [10048]
PASS: Content Cacheability [10049]

```

PS C:\Users\shariq abbasi\Desktop\inventory manage
oxy:stable zap-api-scan.py -t http://host.docker.i
PASS: Cross Site Scripting (Persistent) - Prime [4
PASS: Cross Site Scripting (Persistent) - Spider [
PASS: SQL Injection [40018]
PASS: SQL Injection - MySQL (Time Based) [40019]
PASS: SQL Injection - Hypersonic SQL (Time Based)
PASS: SQL Injection - Oracle (Time Based) [40021]
PASS: SQL Injection - PostgreSQL (Time Based) [400
PASS: Cross Site Scripting (DOM Based) [40026]
PASS: SQL Injection - MsSQL (Time Based) [40027]
PASS: ELMAH Information Leak [40028]
PASS: Trace.axd Information Leak [40029]
PASS: .htaccess Information Leak [40032]
PASS: .env Information Leak [40034]
PASS: Hidden File Finder [40035]
PASS: Spring Actuator Information Leak [40042]
PASS: Log4Shell [40043]
PASS: Exponential Entity Expansion (Billion Laughs
PASS: Spring4Shell [40045]
PASS: Remote Code Execution (React2Shell) [40048]
PASS: Script Active Scan Rules [50000]
PASS: Script Passive Scan Rules [50001]
PASS: Path Traversal [6]
PASS: Remote File Inclusion [7]

```

8.2 Manual Pentest Scenario Matrix

Manual security scenarios were tested to confirm that implemented defenses work correctly.

Vulnerability Target	Injected Payload / Attack	Defense Mechanism	Expected Response
SQL Injection	Username: ' OR '1'='1 --	Zod schema validation and input filtering	400 Bad Request
Cross-Site Scripting	Product name: <script>alert(1)</script>	Zod schema blocks script tags	400 Bad Request
Privilege Escalation	Staff user sends DELETE request to product endpoint	RBAC validates token role	403 Forbidden
Denial of Service	Request body greater than 10kb	Express JSON body size limit	413 Payload Too Large
Unauthorized Access	Request without JWT token	Authentication middleware	401 Unauthorized

These results show that the application correctly rejects malicious input and unauthorized actions.

9. Docker and Kubernetes Security Hardening

The application was containerized and deployed using Kubernetes with several security controls applied at the image and cluster levels.

9.1 Dockerfile Security

The Docker image is built using:

```
node:20-alpine
```

This base image is lightweight and reduces unnecessary packages, which helps reduce the attack surface.

The Dockerfile also creates a non-root application user:

```
appuser
```

The container runs using:

```
USER appuser
```

Running containers as non-root is an important security practice because it limits the impact of container compromise.

```
Dockerfile x
docker > Dockerfile
1 FROM node:20-alpine
2
3 # Set node environment
4 ENV NODE_ENV=production
5
6 WORKDIR /app
7
8 # Create a non-root group and user with safe UID/GID → Tab to Jump
9 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
10
11 # Copy package descriptors
12 COPY package*.json ./
13
14 # Install only production dependencies cleanly
15 RUN npm ci --omit=dev
16
17 # Copy source code, setting ownership to our non-root user
18 COPY --chown=appuser:appgroup . .
19
20 # Switch to non-root execution
21 USER appuser
22
23 EXPOSE 3000
24
25 CMD ["node", "server.js"]
26
```

9.2 Production Dependency Installation

The Docker build installs only production dependencies:

```
npm ci --omit=dev
```

This reduces unnecessary development packages inside the final image and lowers the possibility of dependency-related vulnerabilities.

9.3 Kubernetes Security Context

The Kubernetes deployment applies a restrictive security context:

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
```

These settings ensure that:

- The container does not run as root.

- Privilege escalation is blocked.
- Linux capabilities are removed unless explicitly required.
- The pod follows least-privilege execution principles.

9.4 Kubernetes Secrets Management

Sensitive values such as database connection strings, JWT secrets, and service credentials are stored separately in Kubernetes Secret resources.

This avoids hardcoding sensitive data directly in application code or deployment manifests.

9.5 Kubernetes Network Policy

A Kubernetes NetworkPolicy is used to restrict traffic to only the required services and ports.

Allowed traffic includes:

Service	Port
Application	3000
DNS	53
MongoDB	27017
Redis	6379
Kafka	9092

The policy blocks unnecessary external communication and reduces lateral movement risk inside the cluster.

10. CIS Docker and Kubernetes Compliance Checklist

The following checklist summarizes the implemented security controls against common Docker and Kubernetes hardening expectations.

Control / Standard	Status	Verification Details
Non-root container execution	Implemented	Dockerfile uses <code>USER appuser</code> before runtime execution
Minimal container image	Implemented	Application uses <code>node:20-alpine</code>
Production-only dependencies	Implemented	Docker build uses <code>npm ci --omit=dev</code>
Credential separation	Implemented	Secrets are stored in <code>k8s/secret.yaml</code>
Kubernetes resource limits	Implemented	Deployment sets CPU and memory limits
Liveness probe	Implemented	Kubernetes probes check <code>/health</code> endpoint
Readiness probe	Implemented	Kubernetes readiness check confirms app availability

Network policy isolation	Implemented	NetworkPolicy restricts ingress and egress traffic
Privilege escalation disabled	Implemented	allowPrivilegeEscalation: false
Linux capabilities dropped	Implemented	capabilities.drop: ["ALL"]

These controls demonstrate that the application follows secure container and Kubernetes deployment practices.

11. Jest Integration Test Results

Automated integration testing was performed using Jest. The tests verify authentication, authorization, input validation, product operations, RBAC enforcement, audit log access, security alert access, and OIDC mock identity behavior.

The test command used was:

```
npm run test
```

The test execution produced the following result:

```
> inventory-management-ssdd-final@1.0.0 test
> jest --runInBand --detectOpenHandles --forceExit

console.log
  Kafka Producer connected.

console.log
  MongoDB connected: mongodb://localhost:27017/inventoryhub

PASS ./server.test.js
  InventoryHub Secure API Suite (In-Memory Verification)

    POST /api/auth/register
      ✓ should successfully register an admin user when authorized as admin
      ✓ should successfully register a staff user when authorized as admin
      ✓ should reject registration attempts without authorization header
      ✓ should reject registration attempts by a staff user
      ✓ should reject registration attempts with XSS script tags when
authorized

    POST /api/auth/login
      ✓ should login admin successfully and return a JWT token
      ✓ should login staff successfully and return a JWT token
      ✓ should reject invalid login credentials

    POST /api/products
      ✓ should allow admin to create a new product
      ✓ should reject product creation with invalid schema data
      ✓ should reject product creation containing suspected XSS / script
tags

    DELETE /api/products/:id
      ✓ should deny staff from deleting a product
```

✓ should allow admin to successfully delete a product

Admin-Only RBAC Endpoints

✓ should deny staff from accessing audit logs
✓ should allow admin to view audit logs
✓ should deny staff from accessing security alerts
✓ should allow admin to view security alerts

OIDC Mock Identity Integration

✓ should issue a valid OIDC JWT token and access products successfully
✓ should reject OIDC token request without username or role

Test Suites: 1 passed, 1 total
Tests: 19 passed, 19 total
Snapshots: 0 total
Time: 2.598 s
Ran all test suites.

The successful result confirms that all 19 integration tests passed.

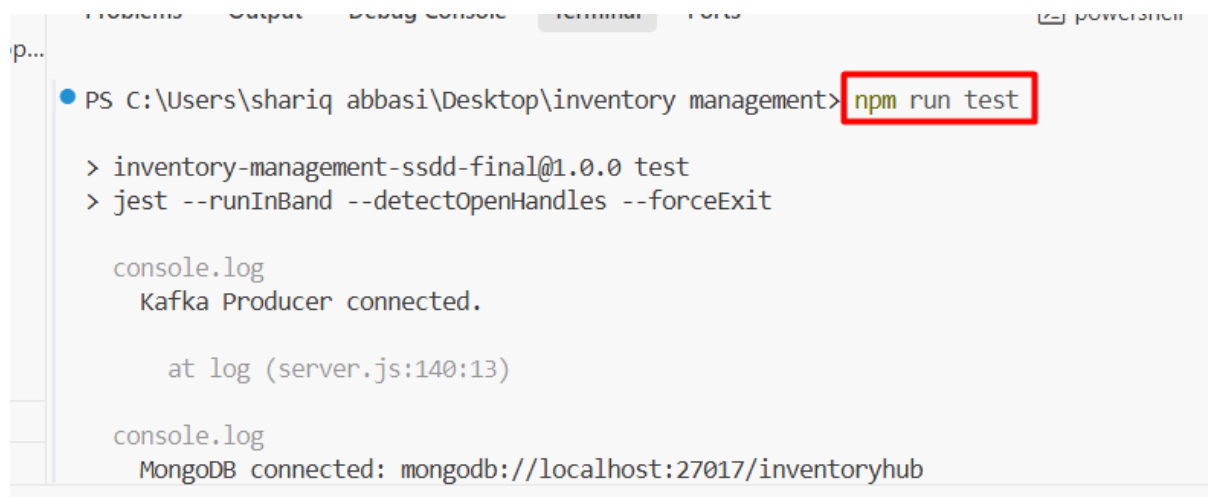
12. Screenshot Evidence

This section contains screenshot placeholders for final submission evidence.

Jest Test Pass Output

Evidence Description:

This screenshot shows the terminal output after running the Jest test suite. It confirms that all integration tests passed successfully.



```
PS C:\Users\shariq abbasi\Desktop\inventory management> npm run test

> inventory-management-ssdd-final@1.0.0 test
> jest --runInBand --detectOpenHandles --forceExit

console.log
  Kafka Producer connected.

    at log (server.js:140:13)

console.log
  MongoDB connected: mongodb://localhost:27017/inventoryhub
```

PASS ./server.test.js

InventoryHub Secure API Suite (In-Memory Verification)

POST /api/auth/register

- ✓ should successfully register an admin user when authorized as admin (137 ms)
- ✓ should successfully register a staff user when authorized as admin (133 ms)
- ✓ should reject registration attempts without authorization header (21 ms)
- ✓ should reject registration attempts by a staff user (163 ms)
- ✓ should reject registration attempts with XSS script tags when authorized (22 ms)

POST /api/auth/login

- ✓ should login admin successfully and return a JWT token (134 ms)

- ✓ should allow admin to view security alerts (19 ms)

OIDC Mock Identity Integration

- ✓ should issue a valid OIDC JWT token and access products successfully (62 ms)
- ✓ should reject OIDC token request without username or role (19 ms)

Test Suites: 1 passed, 1 total

Tests: 19 passed, 19 total

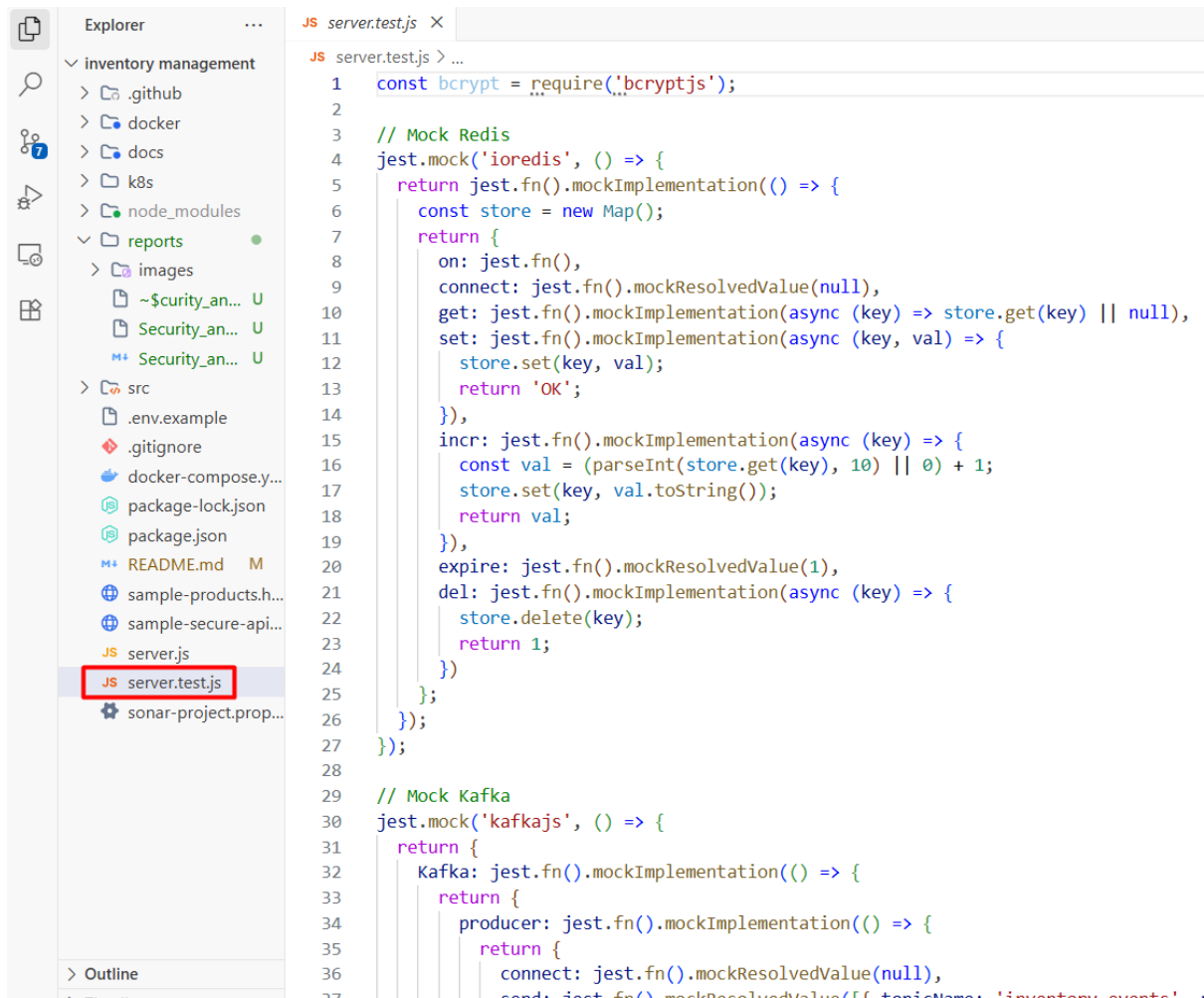
Snapshots: 0 total

Time: 3.022 s

Ran all test suites.

PS C:\Users\shariq_abbasi\Desktop\inventory management>

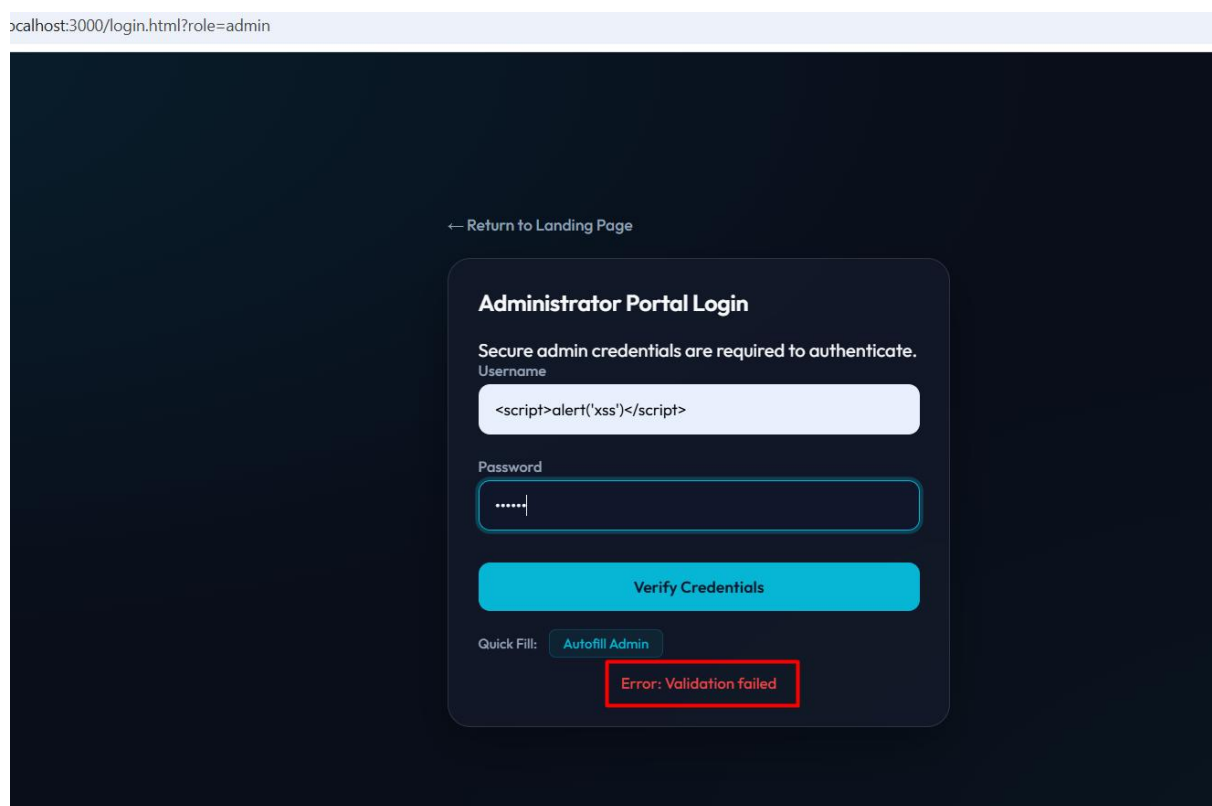
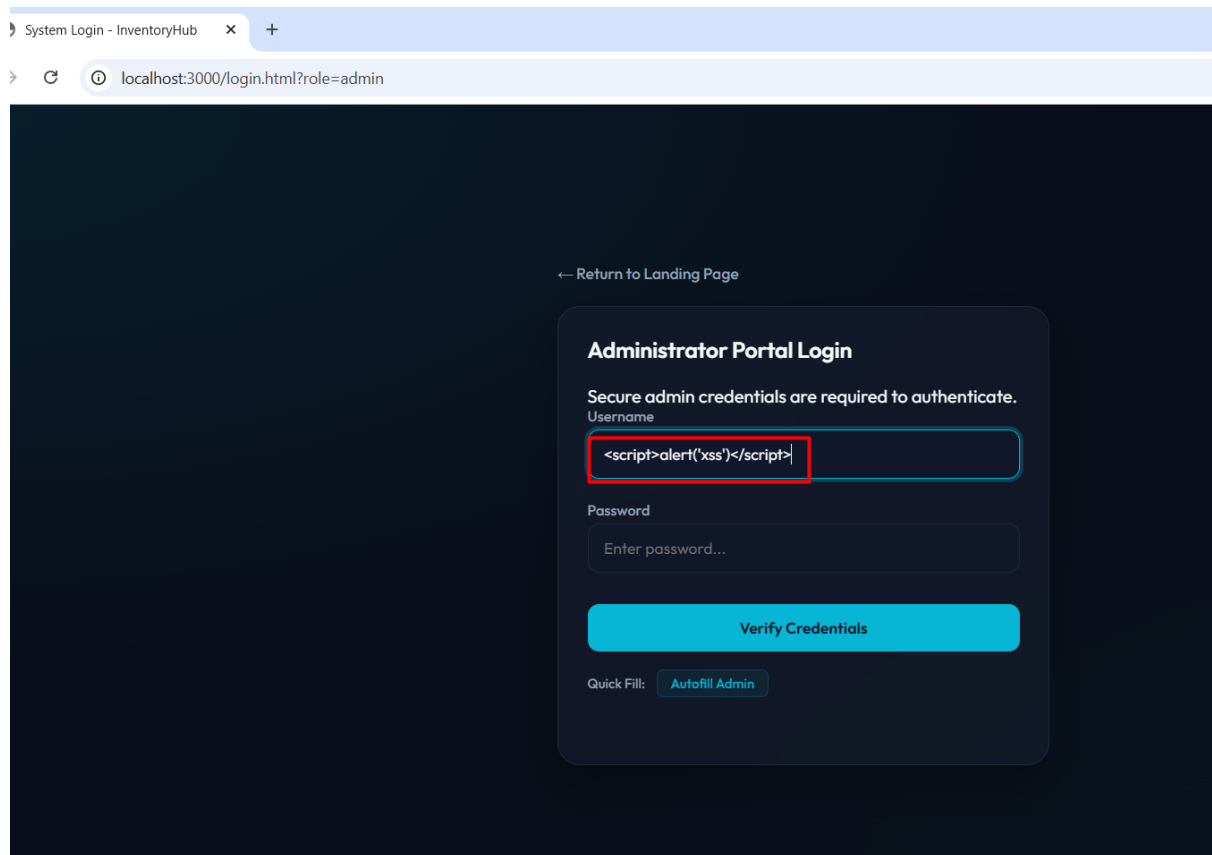
Test cases that were written are:



Input Validation Blocking XSS Payload

Evidence Description:

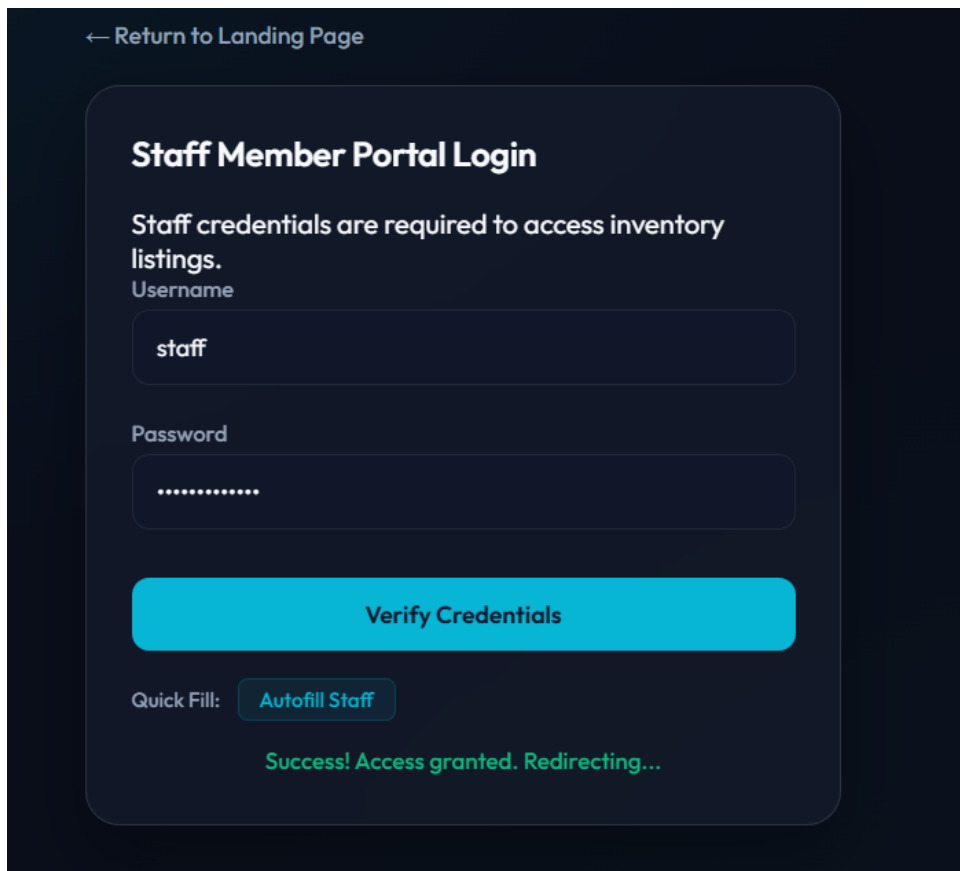
This screenshot demonstrates that the application blocks malicious script input such as `<script>alert(1)</script>`. The expected result is a validation error or a 400 Bad Request response.



Staff Role Blocked from Product Deletion

Evidence Description:

This screenshot proves that a staff user cannot delete a product. The expected result is a 403 Forbidden response or an error message from the interface.



← Return to Landing Page

Staff Member Portal Login

Staff credentials are required to access inventory listings.

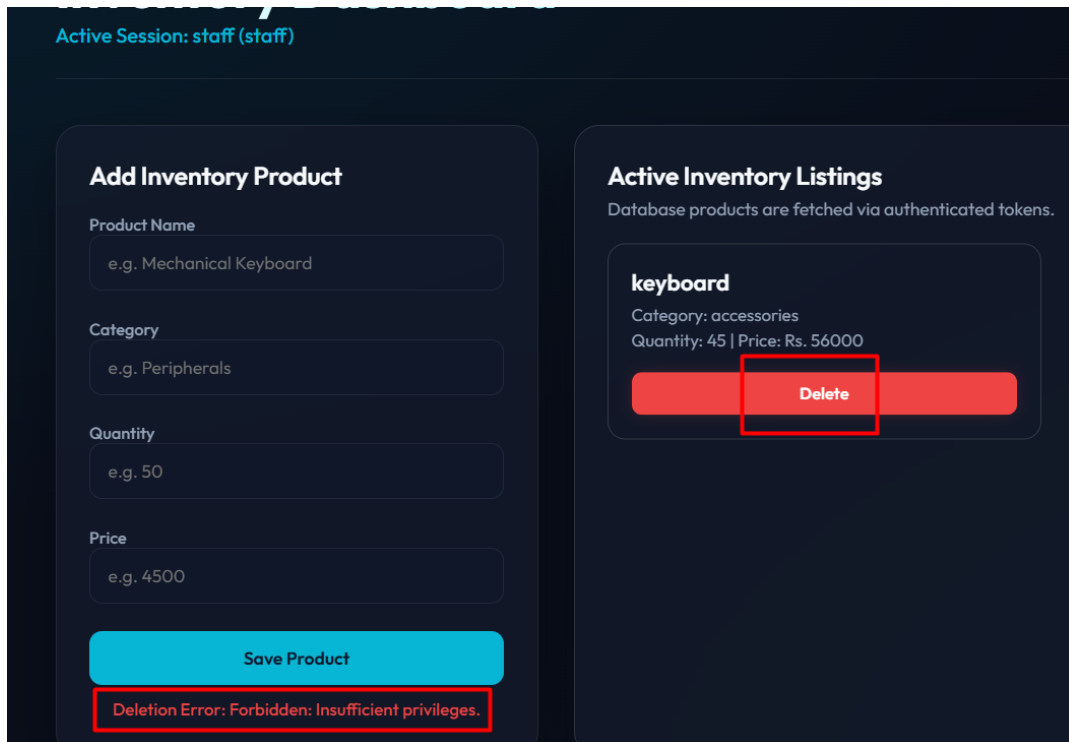
Username

Password

Verify Credentials

Quick Fill: [Autofill Staff](#)

Success! Access granted. Redirecting...



Active Session: staff (staff)

Add Inventory Product

Product Name

Category

Quantity

Price

Save Product

Deletion Error: Forbidden: Insufficient privileges.

Active Inventory Listings

Database products are fetched via authenticated tokens.

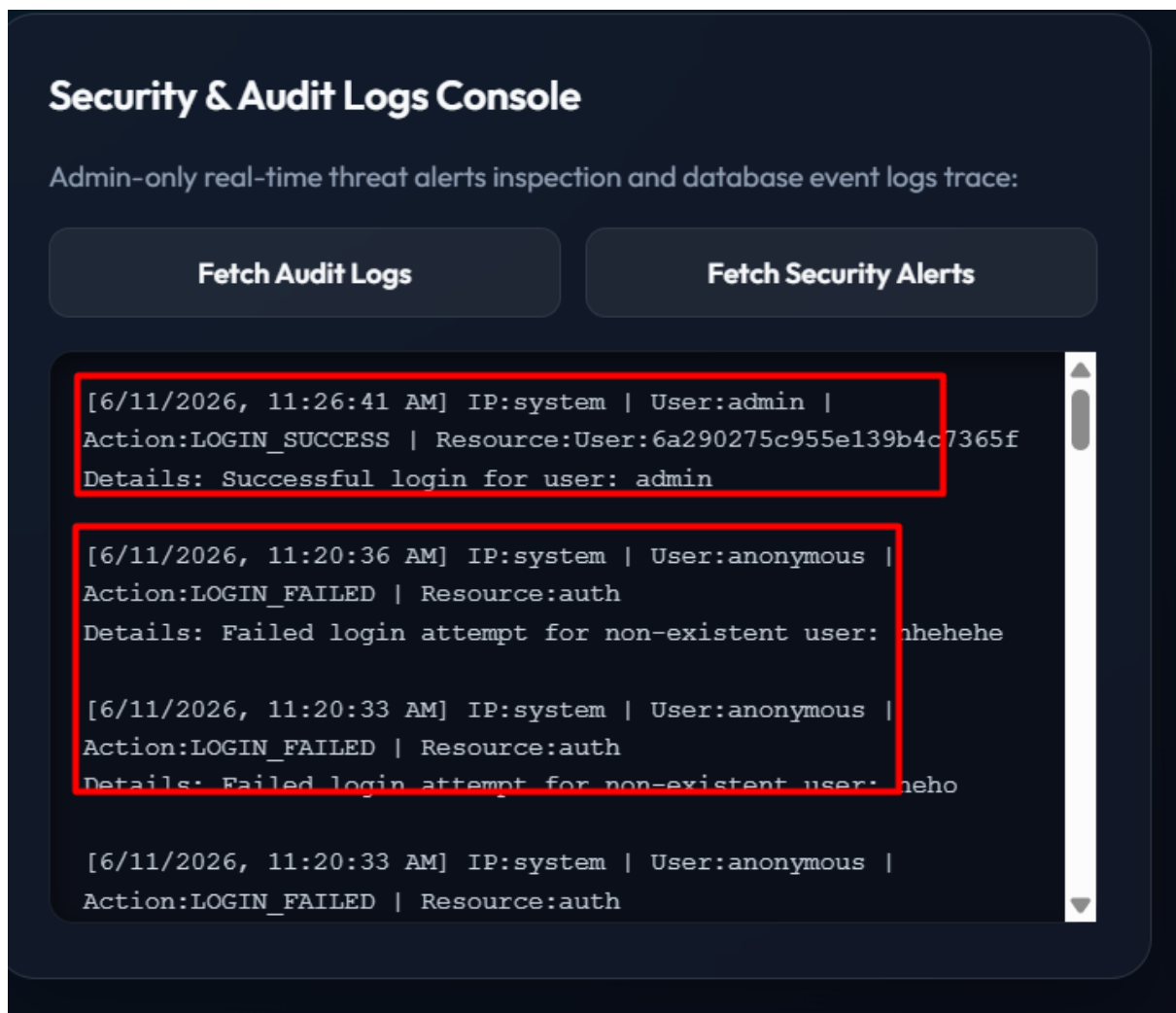
keyboard Category: accessories Quantity: 45 Price: Rs. 56000

Delete

Admin Security Alerts and Audit Logs Dashboard

Evidence Description:

This screenshot shows that the admin dashboard displays audit logs and security alerts. It demonstrates visibility into important system events and suspicious activity.



Trivy Container Vulnerability Scan

Evidence Description:

This screenshot provides evidence of container vulnerability scanning. It should show the Trivy scan result for the InventoryHub Docker image.

Total: 15 (UNKNOWN: 0, LOW: 2, MEDIUM: 2, HIGH: 11, CRITICAL: 0)

Library Fixed Version	Vulnerability	Severity	Status	Installed Version	Fixed Version
brace-expansion (package.json) 1.1.3	CVE-2026-33750 brace-expansion: Denial of Service via zero step value in brace pattern...	MEDIUM	fixed	2.0.1	5.0.5, 3.0.2, 2.0.3, 1.1.1
	https://avd.aquasec.com/nvd/cve-2026-33750				
brace-expansion (package.json) 2.0.1	CVE-2025-5889 brace-expansion: juliangruber brace-expansion index.js expand redos	LOW			2.0.2, 1.1.12, 3.0.1, 4.0.1
	https://avd.aquasec.com/nvd/cve-2025-5889				
cross-spawn (package.json)	CVE-2024-21538 cross-spawn: regular expression denial of service	HIGH		7.0.3	7.0.5, 6.0.6
	https://avd.aquasec.com/nvd/cve-2024-21538				
diff (package.json) 5.1.1	CVE-2026-24001 jsdiff: denial of service vulnerability in parsePatch and parsePatchOptions	LOW		5.2.0	8.0.3, 5.2.2, 4.0.4, 3.0.1
	https://avd.aquasec.com/nvd/cve-2026-27904				
tar (package.json)	CVE-2026-23745 node-tar: tar: node-tar: Arbitrary file overwrite and symlink poisoning via unsanitized linkpaths...			6.2.1	7.5.3
	https://avd.aquasec.com/nvd/cve-2026-23745				
	CVE-2026-23950 node-tar: tar: node-tar: Arbitrary file overwrite via Unicode path collision race condition...				7.5.4
	https://avd.aquasec.com/nvd/cve-2026-23950				
	CVE-2026-24842 node-tar: tar: node-tar: Arbitrary file creation via path				7.5.7

Kubernetes Resources and Network Policy Status

Evidence Description:

This screenshot confirms that Kubernetes resources are deployed successfully. It should show running pods, services, and active network policies.


```
secret/inventory-hub-secrets unchanged
PS C:\Users\shariq abbasi\Desktop\inventory management> kubectl get pods,svc,netpol
>>
NAME                                READY   STATUS    RESTARTS   AGE
pod/inventory-hub-deployment-cd4db857b-42fv1    0/1     Running   0           8s
pod/inventory-hub-deployment-cd4db857b-dpb9d    0/1     Running   0           8s
pod/inventory-hub-deployment-cd4db857b-n5wgc    0/1     Running   0           8s
pod/kafka-deployment-86bb4df4f4-g7627          1/1     Running   0           8s
pod/mongo-deployment-8758df447-7tqbp           1/1     Running   0           8s
pod/redis-deployment-765b75cc7-tkgcf           1/1     Running   0           8s
pod/zookeeper-deployment-697f7848f7-x54z4       1/1     Running   0           8s
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/inventory-hub-service	NodePort	10.109.142.163	<none>	80:30081/TCP	8s
service/kafka-service	ClusterIP	10.96.199.220	<none>	9092/TCP	8s
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	11m
service/mongo-service	ClusterIP	10.99.220.46	<none>	27017/TCP	8s
service/redis-service	ClusterIP	10.102.111.103	<none>	6379/TCP	8s
service/zookeeper-service	ClusterIP	10.104.140.251	<none>	2181/TCP	8s

service/zookeeper-service	ClusterIP	10.104.140.251	<none>	2181/TCP	8s
---------------------------	-----------	----------------	--------	----------	----

NAME	POD-SELECTOR	AGE
networkpolicy.networking.k8s.io/inventory-hub-network-policy	app=inventory-hub	8s

```
PS C:\Users\shariq abbasi\Desktop\inventory management>
```

Rate limiting:

Evidence Description:

This screenshot shows that when there are too many invalid login attempts the system limits the user to login, this limit is for 15 minutes to ensure no brute force attack occurs.

The screenshot displays the 'Administrator Portal Login' interface. At the top, it states 'Secure admin credentials are required to authenticate.' Below this are input fields for 'Username' (containing 'admin') and 'Password' (masked with dots). A prominent blue button labeled 'Verify Credentials' is positioned below the password field. Underneath the button is a 'Quick Fill:' section with an 'Autofill Admin' button. At the bottom, a red-bordered box contains an error message: 'Error: Too many login or registration attempts, please try again after 15 minutes.'

13. Conclusion

The InventoryHub project successfully implements security controls required for Phases 3 and 4 of the Secure Software Development lab. The application includes strong authentication, password hashing, role-based access control, request validation, audit logging, security alerting, rate limiting, and secure middleware configuration.

Static analysis tools such as CodeQL and SonarQube support secure code review, while dynamic testing through OWASP ZAP and manual pentest scenarios confirms runtime protection against common attacks. The Docker and Kubernetes deployment configurations follow important hardening practices, including non-root execution, secret separation, resource limits, readiness probes, liveness probes, and network policy isolation.

The Jest test results confirm that the implemented security features work correctly, with all 19 integration tests passing successfully. Overall, InventoryHub demonstrates a secure, tested, and compliance-aware application suitable for final project submission.